



Operating system and system programming (25CS2104E)

Multithreaded Linux Application Using POSIX Threads and Mutexes

Review-I

Under the Esteemed Guidance of

Raghupathi M
CSE Department

Team Members: 23

2520090110 - Aditya Vallala

2520080067 - Dheeraj Naidu

2520080054 - Surapaneni karthikeya

Contents

- Abstract
- Introduction
- Hardware and Software requirements
- System Architecture
- Conclusion

Abstract

The Multithreaded Linux Application Using POSIX Threads and Mutexes is designed to demonstrate the concepts of concurrent programming in a Linux environment. The application uses POSIX Threads (Pthreads) to execute multiple tasks concurrently, improving responsiveness and resource utilization. Since multiple threads may access shared resources simultaneously, mutexes (mutual exclusion locks) are used to prevent race conditions and maintain data consistency. The project demonstrates thread creation, execution, synchronization, shared-resource management, and proper thread termination. It provides a practical understanding of how multithreading and synchronization mechanisms are implemented in Linux using the C programming language.

Introduction

Multithreading is an important concept in modern operating systems that allows a single process to perform multiple tasks concurrently. A thread is a lightweight unit of execution within a process, and multiple threads can share the same memory and resources. This makes multithreading useful for applications that need better performance, responsiveness, and efficient CPU utilization. Linux provides strong support for multithreaded programming through the POSIX Threads (Pthreads) library. Functions such as `pthread_create()`, `pthread_join()`, and `pthread_exit()` are used to create, manage, and terminate threads. When multiple threads access a common resource, problems such as race conditions can occur. To avoid these problems, the project uses mutexes. A mutex allows only one thread at a time to access a critical section of code. This ensures that shared data is modified safely and consistently. The project therefore focuses on implementing a Linux application that demonstrates thread creation, concurrent execution, synchronization using mutexes, critical-section protection, and safe resource management.

Hardware and Software requirements

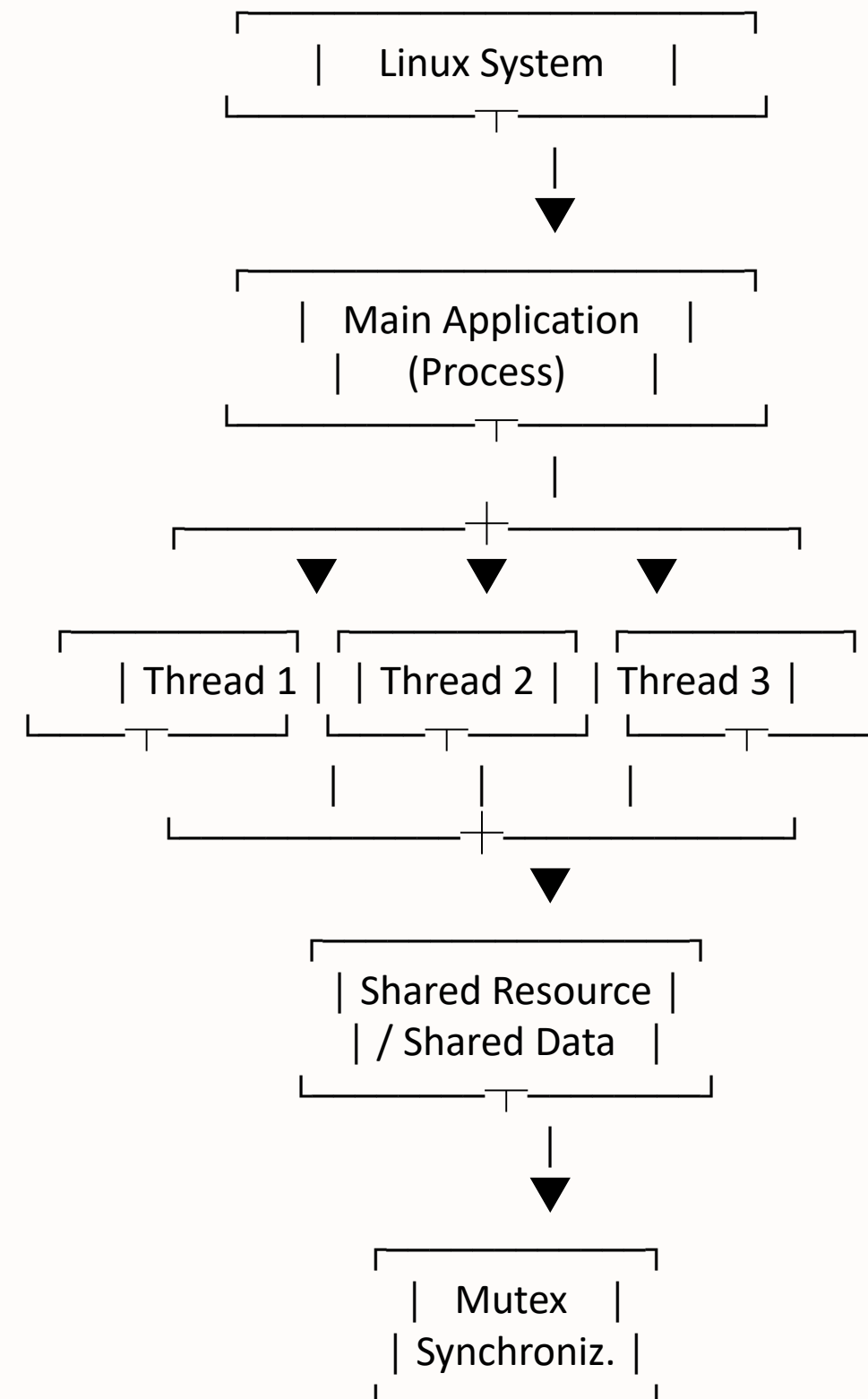
Hardware requirements :

Component	Requirement
Processor	Intel/AMD Dual-Core or above
RAM	Minimum 4 GB
Storage	Minimum 10 GB free space
Keyboard	Standard keyboard
Display	Standard monitor
Network	Optional

Software requirements :

Software	Requirement
Operating System	Linux (Ubuntu recommended)
Programming Language	C
Compiler	GCC
Thread Library	POSIX Threads (Pthreads)
Development Tools	GCC, Make, Text Editor/IDE
Terminal	Linux Terminal
Libraries	pthread.h, standard C libraries

System Architecture



Conclusion

The Multithreaded Linux Application Using POSIX Threads and Mutexes successfully demonstrates the fundamental concepts of multithreaded programming in Linux. The project shows how multiple threads can execute concurrently and how POSIX Threads can be used to create and manage them. The use of mutexes provides safe access to shared resources and prevents race conditions and data inconsistency. Overall, the project provides practical knowledge of thread creation, concurrent execution, synchronization, critical sections, mutex locking and unlocking, and thread termination. These concepts are essential for developing efficient and reliable Linux applications and form an important foundation for advanced operating-system and concurrent-programming applications.

Thank you